Operating Systems
Week 6 Lecture 6

Junaid Abdul Wahid

Previous

- SJF, Preemptive SJF

Today Agenda

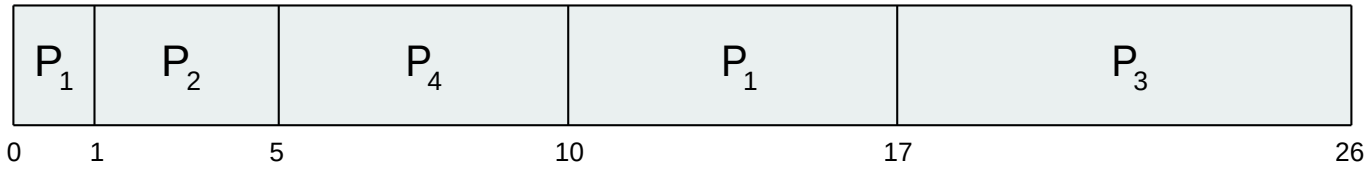
- Scheduling Algorithms Continued

Example of Shortest-remaining-time-first

Now we add the concepts of varying arrival times and preemption to the analysis

Process	*Arrival* Time	Burst Time
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

Preemptive SJF Gantt Chart



Average waiting time = [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5

Shortest-remaining-time-first

Compare all processes in the ready queue: At every decision point, compare the remaining burst times of all processes in the **ready queue** (not just the newly arrived process).

At every step, the scheduler compares the remaining burst times of all processes in the ready queue (including the currently running process, if any).

Preemption: If a new process has a shorter remaining burst time than the currently running process, preempt the current process and schedule the new one.

Shortest-remaining-time-first

No priority for "upper" processes: The order of processes in the table (e.g., P1, P2, P3, P4) does not matter. The decision is purely based on the shortest remaining burst time.

Waiting Time Explanation

Waiting Time=Start Time−Arrival Time

where:

- **Start Time** = Time when the process starts executing.
- **Arrival Time** = Time when the process arrives in the ready queue.

For preemptive algorithms like SRTF, the waiting time includes all the intervals during which the process is in the ready queue but not executing.

Waiting Time Explanation

Waiting Time=Total time spent waiting in the ready queue

•**P1**: Waits from time 1 to time 10 → 10 - 1 = 9. arrives at 0 starts executing until 1, then preempted at time 1 and return to ready queue, resumes execution at 10, Total waiting time spent in ready queue.

•**Waiting Time for P1=Time when P1 resumes execution−Time when P1 was preempted**
Waiting Time for P1=10−1=9Waiting Time for P1=10−1=9

•**P2**: No waiting time → 1 - 1 = 0. Arrives at 1 and starts executing immediately

•**P3**: Waits from time 2 to time 17 → 17 - 2 = 15.

•**P4**: Waits from time 3 to time 5 → 5 - 3 = 2.

Round Robin (RR)

Each process gets a small unit of CPU time (**time quantum** q), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.

If there are n processes in the ready queue and the time quantum is q, then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.

Timer interrupts every quantum to schedule next process

Performance

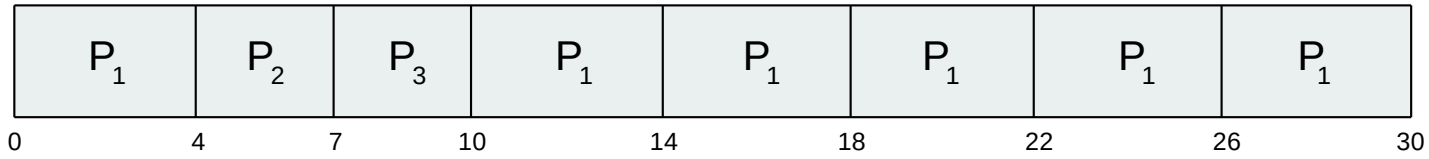
- q large $\Rightarrow$ FIFO (FCFS)
- q small $\Rightarrow$ RR

Note that q must be large with respect to context switch, otherwise overhead is too high。

Example of RR with Time Quantum = 4

Process	Burst Time
P_1	24
P_2	3
P_3	3

The Gantt chart is:

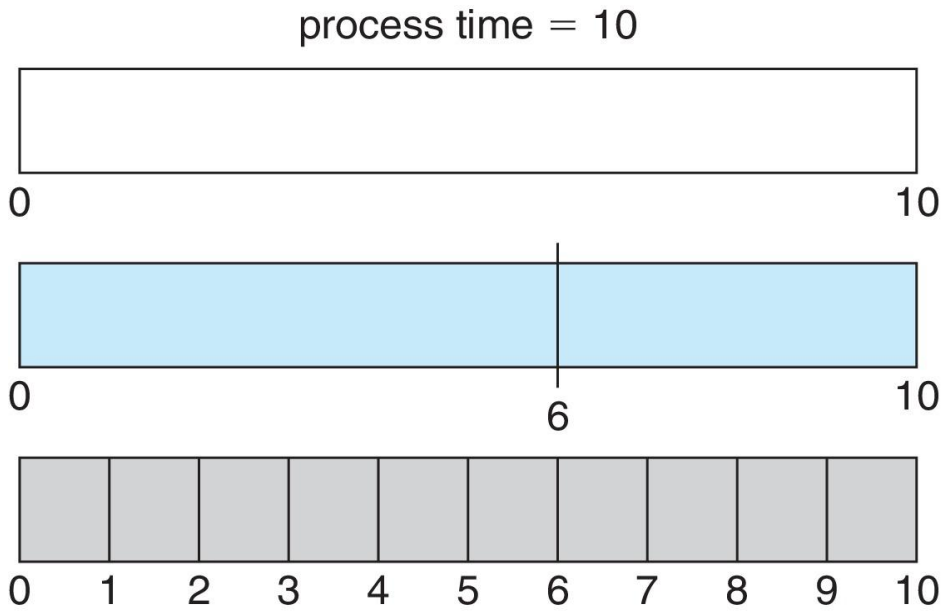


Typically, higher average turnaround than SJF, but a better ***response***

q should be large compared to context switch time

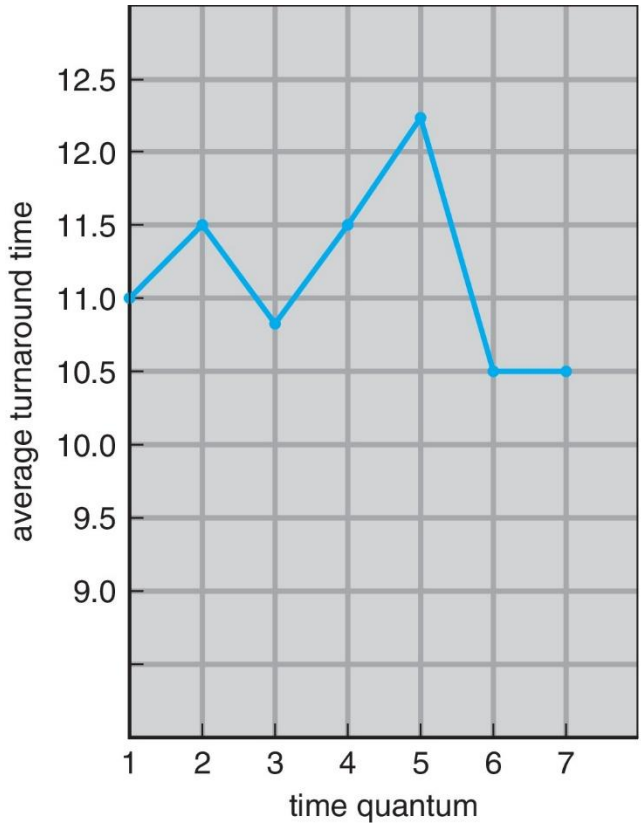
- q usually 10 milliseconds to 100 milliseconds,
- Context switch < 10 microseconds

Time Quantum and Context Switch Time



quantum	context switches
12	0
6	1
1	9

Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts should be shorter than q

Priority Scheduling

A priority number (integer) is associated with each process

The CPU is allocated to the process with the highest priority (smallest integer ≡ highest priority)

- Preemptive
- Nonpreemptive

SJF is priority scheduling where priority is the inverse of predicted next CPU burst time

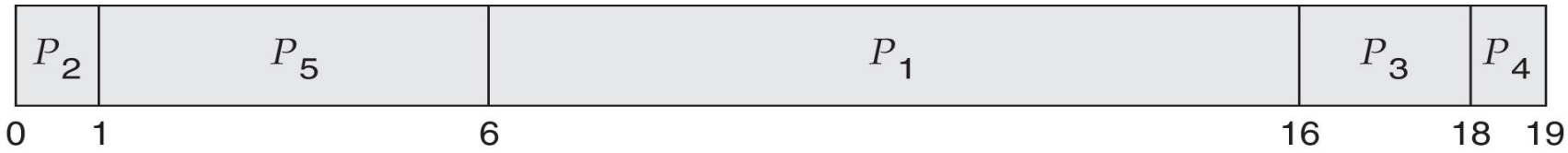
Problem ≡ **Starvation** – low priority processes may never execute

Solution ≡ **Aging** – as time progresses increase the priority of the process

Example of Priority Scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

Priority scheduling Gantt Chart



Average waiting time = 8.2